DOCKER / FIELD GUIDE

Docker, made teachable

A visual study companion to Nigel Poulton's Docker Deep Dive, May 2020 edition.

Build. Ship. Run.

Understand the model. Practice the commands. Explain the ideas.



The central workflow: build an image, distribute it, and create containers from it.

Inside this guide

LEARN	PRACTICE	TEACH
Short definitions and core concepts	Runnable command examples	Diagrams and comparison tables
Images, lifecycle, builds and Compose	Networking and persistence labs	Questions with answer keys
Swarm, stacks and security	Two-page command reference	A ready-to-use lesson outline

How to use it
Read pages 2-8 for the foundation. Work through pages 9-16 for applications and operations. Keep pages 17-18 nearby while practicing; use pages 19-22 for troubleshooting, labs and teaching.

Editorial approach: original, condensed explanations and new examples, rather than copied book passages. Book page numbers refer to printed page labels, not the PDF viewer count. Modernization notes and links are on page 23.

Your learning route

Move from individual containers to complete applications and distributed services.

STAGE	YOU SHOULD BE ABLE TO...	GUIDE
Foundations	Explain images, containers, VMs and the Engine.	3-4
Artifacts	Pull, inspect and identify an image.	5
Runtime	Run, inspect, stop and replace a container.	6
Builds	Write a Dockerfile and explain caching.	7-8
Applications	Describe and run an app using Compose.	9
Infrastructure	Connect containers and preserve data.	10-11
Orchestration	Explain Swarm, quorum, overlays and stacks.	12-15
Security	Connect each security control to its purpose.	16
Fluency	Choose commands and diagnose common failures.	17-21

The five ideas to remember

1. Image and container are different. The image is the reusable package; a container is one runtime instance with its own writable layer.

2. A container is a process boundary. Its main process controls its running lifetime; it shares the host kernel.

3. State needs an explicit home. Put valuable data in a volume or another external storage system.

4. Networks have scope. A user-defined bridge joins containers on one host; an overlay can span Swarm nodes.

5. Desired state drives orchestration. Define the service you want; the orchestrator keeps trying to realize it.

> **Practice conditions**
> Examples assume a Linux-container Docker environment, a POSIX shell, internet access to pull images, and permission to use Docker. They are illustrative and have not been executed against a Docker daemon in this workspace.

Source: book chapters 1-16, printed pp. 7-238.

The essential vocabulary

Use these short definitions when learning or explaining Docker.

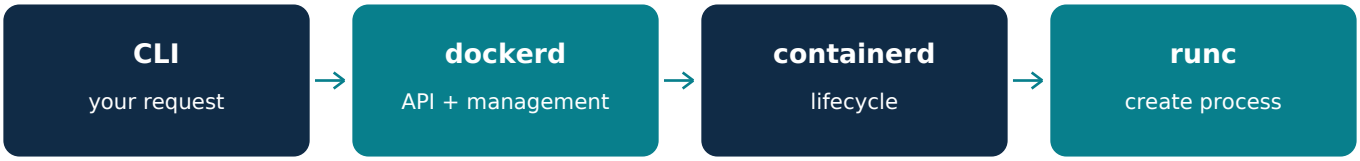
TERM	SHORT DEFINITION	EXAMPLE / MEMORY AID
Docker Engine	The software and API that build and manage container resources.	The service behind the docker CLI.
Image	A read-only application package plus configuration.	nginx:alpine
Container	An instance of an image, with processes and runtime configuration.	web created from nginx:alpine
Dockerfile	A text recipe for building an image.	FROM, COPY, RUN, CMD
Registry	A service that stores and distributes images.	Docker Hub or a private registry.
Repository	A named collection of image versions in a registry.	library/nginx
Tag	A human-readable reference that can move.	alpine or 1.0
Digest	A content-addressed identifier for a manifest or image index.	sha256:...
Volume	Docker-managed storage with a separate lifecycle.	notes mounted at /data
Compose	A declarative way to manage a multi-container app.	compose.yaml
Service / task	Swarm desired-state definition / its scheduled unit.	Three replicas means three tasks.
Stack	A set of related Swarm services and resources.	A web tier plus a data tier.

Explain it in one sentence
Docker packages an application as an image and runs instances of that image as isolated processes, with explicit networking and storage.

Source: book chapters 1-4, 6-7; printed pp. 7-30, 51-90.

What runs under the hood?

A simplified Engine model and a useful comparison with virtual machines.



Simplified Linux execution path. A shim supervises the container process and its I/O; runc does not remain as the long-running supervisor.

Containers and virtual machines

QUESTION	CONTAINER	VIRTUAL MACHINE
What is isolated?	Processes, filesystems and resource views.	A machine with its own guest OS.
Kernel	Shared with the container host.	Each guest has its own kernel.
Typical overhead	Less OS duplication.	A full guest OS per VM.
Compatibility	Image must suit the kernel and CPU platform.	Guest OS must suit the virtual hardware.
Useful together?	Containers can run inside VMs.	VMs can provide host isolation boundaries.

Why standards matter

The OCI image and runtime specifications give tools a common packaging and execution contract. Docker images and container runtimes are connected by these standards; the Docker CLI is only one interface to the ecosystem.

> **Avoid this misconception**
> An image may contain Linux user-space tools and libraries, but it does not bring its own Linux kernel. Running Linux containers on macOS requires a Linux environment behind the scenes.

Recall: Which component accepts the Docker API? Which creates the low-level container process? Answer: dockerd; an OCI runtime such as runc.

Source: book chapters 5 and 7, printed pp. 33-49, 73-90.

Images: the reusable artifact

Name them clearly, inspect them, and distinguish tags from content identity.

DOWNLOAD AND INSPECT

```
docker image pull nginx:alpine
docker image ls
docker image inspect nginx:alpine
docker image history nginx:alpine
```

CONCEPT	WHAT IT MEANS	WHY IT MATTERS
Layers	Read-only filesystem changes assembled into an image.	Shared layers reduce duplicate storage and transfer.
Writable layer	Private changes added when a container is created.	Deleting a container discards this layer.
Tag	A movable name for an image version.	The same tag can resolve differently later.
Digest	A hash-based reference to exact manifest content.	Useful when exact artifact identity matters.
Platform	OS and CPU architecture selected for execution.	A multi-platform reference may select different images.

Name anatomy

```
registry.example.com/team/web:1.0
# registry           repository   tag

# After building web-guide:1.0 (page 7):
docker image tag web-guide:1.0 YOUR_USER/web-guide:1.0
docker login
docker image push YOUR_USER/web-guide:1.0
```

Replace YOUR_USER with your registry namespace. Pushing requires an account and repository permissions. Use a full @sha256:... digest obtained from the registry when pinning exact content.

> **Learning shortcut**
> Tag = convenient label. Digest = content identity. Neither a version-looking tag nor the word latest proves freshness or safety.

Source: book chapter 6, printed pp. 51-72.

The container lifecycle

A container can exist without running; stopping and removing are different operations.



docker run creates and starts. docker stop asks the process to exit. docker start restarts an existing stopped container; docker rm removes it.

ACTION	EXAMPLE	MEANING
Create + start	docker run -d --name web nginx:alpine	Create a new background container.
List all	docker container ls -a	Include stopped containers.
Observe	docker logs --tail 20 web	Read captured output.
Run another process	docker exec -it web sh	Open a shell inside the running container.
Stop / start	docker stop web / docker start web	Reuse the same container and writable layer.
Remove	docker rm web	Delete a stopped container and its writable layer.

Three useful distinctions

run vs start: run creates a new container; start reuses one that already exists. Editing an image does not update existing containers.

exec vs attach: exec launches a separate process. attach connects to the main process streams and can forward signals.

stop vs kill: stop sends the configured stop signal, normally SIGTERM, then forces termination after a timeout. kill normally sends SIGKILL immediately.

> **Restart policies have a narrow job**
> A policy such as unless-stopped can restart an exited container. It does not schedule replicas on other hosts. A standalone container becoming unhealthy does not, by itself, cause a restart.

Source: book chapter 7, printed pp. 73-90.

Build your first image

A small static site demonstrates the complete build-and-run loop.

Create a new empty directory. Save the following two files inside it. The trailing dot in the build command selects that directory as the build context.

FILE: index.html

```
<h1>Hello from my Docker image</h1>
```

FILE: Dockerfile

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

BUILD, TEST AND CLEAN UP

```
docker build -t web-guide:1.0 .
docker run -d --name guide-web \
  -p 127.0.0.1:8080:80 web-guide:1.0
curl http://localhost:8080
docker stop guide-web
docker rm guide-web
```

INSTRUCTION	SHORT DEFINITION	IMPORTANT DISTINCTION
FROM	Choose a base image or begin a stage.	A new stage begins with each FROM.
COPY	Add files from the build context.	Files excluded by .dockerignore are unavailable.
RUN	Execute a build-time command.	It does not run on every container startup.
WORKDIR	Set the working directory.	Applies to later instructions and default execution.
CMD / ENTRYPOINT	Define default runtime behavior.	CMD is commonly overridden; ENTRYPOINT sets the executable.
EXPOSE	Document an intended container port.	It does not publish a host port.

The base image already supplies the web server startup configuration. Binding the published port to 127.0.0.1 keeps this demonstration on the local host.

Source: book chapter 8, printed pp. 91-107. Example created for this guide.

Caching and multi-stage builds

Separate what is needed to build from what is needed to run.

Cache ordering

Builds can reuse cached results. Put stable, expensive work before frequently changing source files. If dependency metadata changes, reinstall dependencies; if only application code changes, try to reuse the dependency step.

ORDER DEPENDENCIES BEFORE SOURCE

```
# Pattern for a Node project with a lockfile:
COPY package*.json ./
RUN npm ci
COPY . .
```

Use .dockerignore to exclude local dependency directories, version-control files, build output, and sensitive files from the build context. Do not copy secrets into image layers.

A minimal multi-stage illustration

BUILD STAGE -> FINAL STAGE

```
FROM alpine AS build
RUN mkdir -p /out && \
    echo "Built in a separate stage" > /out/index.html

FROM nginx:alpine
COPY --from=build /out/index.html \
    /usr/share/nginx/html/index.html
```



In a real build, compilation or asset generation replaces the echo step. Build-only files stay out of the final image unless copied across.

> **What this improves**
> The final artifact can be smaller and contain fewer build tools. Measure real image sizes; multi-stage builds do not guarantee a fixed percentage reduction.

Source: book chapter 8, printed pp. 101-107.

Define an application in YAML

Compose groups containers, networks and volumes into one reproducible project.

Save this as compose.yaml in a dedicated folder. It creates a web server and Redis service. This is an infrastructure demonstration: the static web page does not query Redis.

FILE: compose.yaml

```
services:
  web:
    image: nginx:alpine
    ports:
      - "127.0.0.1:8080:80"
  redis:
    image: redis:alpine
    command: ["redis-server", "--appendonly", "yes"]
    volumes:
      - cache-data:/data
volumes:
  cache-data:
```

RUN FROM THE DIRECTORY CONTAINING THE FILE

```
docker compose config
docker compose up -d
docker compose ps
docker compose exec redis redis-cli ping
docker compose logs --tail 20
docker compose down
```

Read the model

services: describes the containers and their configuration. **ports:** maps a host port to a container port. **volumes:** attaches named storage. Compose creates a project network and service-name discovery by default.

> **Lifecycle and readiness**
> down removes the project containers and networks, preserving named volumes by default. down -v also deletes declared named volumes. Startup order alone does not prove an application is ready; use health checks, dependency conditions where appropriate, and application retries.

Source: book chapter 9, printed pp. 109-125. Current syntax: references [1-2], page 23.

Connect by name, publish by need

A user-defined bridge is the usual starting point on a single Docker host.



Published ports connect clients to a container through the host. A container calling localhost reaches itself, not another service.

NETWORK TYPE	SCOPE / PURPOSE	MEMORY AID
User-defined bridge	Containers on one host; name-based discovery.	A local application network.
Overlay	Communication across Swarm nodes.	A logical network over a physical network.
Host	Share the host network namespace where supported.	No separate container network namespace.
Macvlan	Present containers on an existing layer-2 network.	Each endpoint has its own MAC identity.
None	Disable ordinary container networking.	Only the loopback interface.

TEST SERVICE-NAME REACHABILITY

```
docker network create class-net
docker run -d --name class-web \
  --network class-net nginx:alpine
docker run --rm --network class-net alpine \
  wget -qO- http://class-web
docker network inspect class-net
docker stop class-web
docker rm class-web
docker network rm class-net
```

Port rule
For -p 127.0.0.1:8080:80, 8080 belongs to the host and 80 belongs to the container. Containers on the same bridge can talk directly to listening container ports without publishing them.

Source: book chapter 11, printed pp. 151-173.

Keep data beyond a container

Separate disposable compute from data that must survive replacement.

STORAGE	LIFECYCLE / CONTROL	GOOD FIT
Writable container layer	Tied to that container; removed with it.	Temporary changes that may be discarded.
Named volume	Docker-managed; survives container removal.	Application data that needs persistence.
Bind mount	A chosen host path appears in the container.	Source editing or host-provided configuration.
tmpfs mount	Memory-backed temporary storage on Linux.	Ephemeral data that need not persist.

Prove persistence with two disposable containers

EXPECTED OUTPUT FROM SECOND CONTAINER: Docker lesson

```
docker volume create class-notes
docker run --rm \
  --mount type=volume,src=class-notes,dst=/data \
  alpine sh -c 'echo "Docker lesson" > /data/note.txt'

docker run --rm \
  --mount type=volume,src=class-notes,dst=/data \
  alpine cat /data/note.txt

docker volume inspect class-notes
```

The first container writes a file and is removed. The second container reads the same volume, proving the data outlives either container.

> **Persistence is not backup**
> A volume can still be deleted or corrupted. Back up valuable data and test restoration. A local volume on one Swarm node is not automatically shared or replicated to other nodes.

OPTIONAL CLEANUP: DELETES THE EXERCISE DATA

```
# Only after this exercise data is no longer needed:
docker volume rm class-notes
```

Source: book chapter 13, printed pp. 189-198.

From containers to services

Swarm combines a cluster of Engines with a desired-state orchestration system.

OBJECT	DEFINITION	PRACTICAL EFFECT
Node	An Engine participating in the Swarm.	Can host tasks according to its role and availability.
Manager	Maintains cluster state and schedules work.	Managers use Raft consensus.
Worker	Executes tasks assigned by managers.	Does not vote in manager consensus.
Service	A desired configuration for scheduled tasks.	Defines image, replicas and update policy.
Task	A scheduled unit for one service container.	A failed task is replaced with a new task.
Replicated service	A service with a chosen replica count.	Example: three copies of a web service.
Global service	One task per eligible node.	Useful for per-node agents.

One-node learning lab

RUN ON AN ISOLATED PRACTICE ENGINE

```
docker swarm init
docker node ls
docker service create --name class-site \
  --replicas 3 --publish published=8081,target=80 \
  nginx:alpine
docker service ls
docker service ps class-site
docker service scale class-site=5
```

If init requests an advertised address, supply a reachable host IP with --advertise-addr. A one-node Swarm is useful for learning but has no host-failure tolerance. The join command printed by init is used on additional Engines.

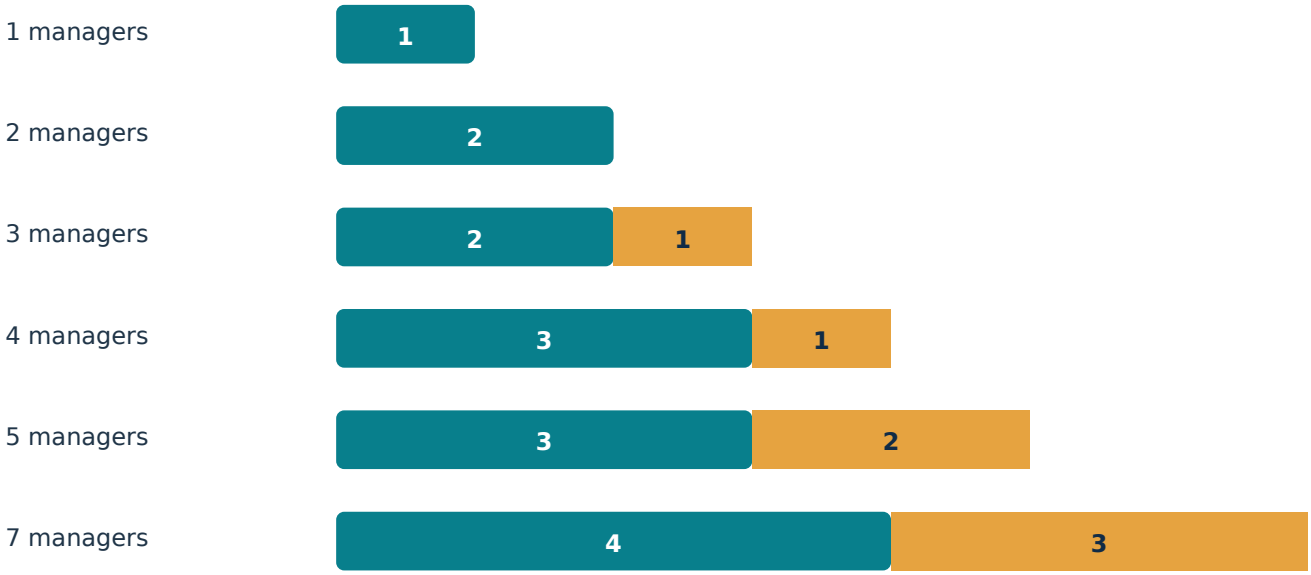
> **Reconciliation is the central idea**
> If desired replicas = 5 and only 4 are running, Swarm tries to schedule a replacement. Capacity, image pulls, constraints or storage failures can prevent convergence.

Source: book chapter 10, printed pp. 127-149.

Quorum: count the managers

More worker nodes add execution capacity; manager count determines consensus tolerance.

A majority of managers must be available to change cluster state. With N managers, quorum is floor(N / 2) + 1; tolerated manager failures are N - quorum.



TEAL = managers required for quorum. GOLD = manager failures tolerated. Counts describe consensus, not application availability.

Why odd manager counts are useful

Three managers tolerate one failure; four also tolerate only one. Five tolerate two. An even count can add coordination work without increasing failure tolerance relative to the preceding odd count.

> **Losing quorum**
> Existing tasks may keep running, but the control plane cannot reliably make changes or schedule replacements until quorum is restored. Application availability also depends on workers, networking and storage.

Updates and recovery

```
docker service update --image nginx:alpine \
  --update-parallelism 1 --update-delay 10s class-site
docker service rollback class-site
```

The update command shows the syntax. To observe an actual version rollout, choose a different verified image reference; reusing an unchanged image may produce no replacement. Back up Swarm control-plane state and application data separately.

Source: book chapter 10, printed pp. 132-135.

A network that spans hosts

Overlay networking makes remote container endpoints appear on one logical network.



Conceptual packet path: container traffic is encapsulated, carried between hosts, and delivered to a remote endpoint.

Short definitions

TERM	MEANING
Underlay	The real IP network connecting the Docker hosts.
Overlay	A logical network built on top of the underlay.
VXLAN	Encapsulation that carries layer-2 frames over a layer-3 network.
VTEP	A tunnel endpoint that encapsulates or decapsulates VXLAN traffic.
VNI	A VXLAN identifier used to distinguish logical networks.
Control plane	Distributes network membership and configuration.
Data plane	Carries the application traffic itself.

REQUIRES SWARM MODE

```
docker network create --driver overlay \
  --attachable class-overlay
docker service create --name overlay-web \
  --network class-overlay --replicas 2 nginx:alpine
docker network inspect class-overlay
```

To demonstrate cross-host communication, add a second reachable Engine to the Swarm and ensure endpoints actually run on different nodes. A one-host test demonstrates setup, not a cross-host tunnel.

> **Security distinction**
> Swarm management traffic is protected with TLS. Do not assume application overlay traffic is encrypted just because the control plane is secure. Encrypted overlay options have platform constraints and must be configured explicitly.

Source: book chapter 12, printed pp. 175-187.

Deploy the whole Swarm app

A stack describes a group of services and supporting resources declaratively.

Save this separate example as stack.yaml. Run stack commands on a manager. Stack deployment uses the legacy Compose v3 format; it is not interchangeable with every modern Compose feature.

FILE: stack.yaml

```
version: "3.8"
services:
  web:
    image: nginx:alpine
    ports:
      - "8082:80"
    deploy:
      replicas: 3
      update_config:
        parallelism: 1
        delay: 10s
      restart_policy:
        condition: on-failure
```

DEPLOY, OBSERVE, REMOVE

```
docker stack deploy -c stack.yaml class-app
docker stack ls
docker stack services class-app
docker stack ps class-app
docker stack rm class-app
```

COMPOSE	SWARM STACK
Manages an application on an Engine.	Manages services across Swarm nodes.
docker compose up -d	docker stack deploy -c stack.yaml NAME
Can build local project images.	Build and push images first; nodes need access.
Uses the current Compose Specification.	Uses a legacy v3 deployment format.

Keep desired configuration in source control. Edit the stack file and redeploy it when changing replicas or image references; ad hoc changes can be overwritten by the next declarative deployment.

Source: book chapter 14, printed pp. 199-216. Compatibility: reference [3], page 23.

Match the control to the risk

Security comes from several boundaries working together.

CONTROL	SHORT DEFINITION	WHAT IT HELPS WITH
Namespaces	Isolate views of processes, networking and other resources.	Limits what a process can see.
cgroups	Account for and constrain resource use.	Reduces resource exhaustion impact.
Capabilities	Split root privileges into smaller permissions.	Allows dropping unneeded privileges.
seccomp	Restrict permitted system calls.	Reduces exposed kernel interfaces.
AppArmor / SELinux	Apply mandatory access-control policies.	Constrain file and resource access.
TLS / mutual TLS	Encrypt transport and authenticate endpoints.	Protects remote API and cluster communication.
Secrets	Deliver sensitive values separately from images.	Avoids baking credentials into artifacts.
Scanning / signing	Check known issues / verify artifact provenance.	Supports supply-chain review; neither proves safety.

One constrained Linux-container example

```
docker run --rm --user 65534:65534 \
  --read-only --cap-drop ALL \
  --security-opt no-new-privileges=true \
  --memory 128m --cpus 0.5 \
  alpine id
```

> **Treat Docker access as privileged**
> Access to a privileged Docker daemon can grant host-level control. Restrict daemon/socket access, secure remote API connections, avoid unnecessary privileged containers, and keep secrets outside image layers.

The book covers Docker Content Trust / Notary v1. Treat that implementation as historical: Docker documents its retirement. See reference [4] on page 23 for migration guidance.

Source: book chapters 5 and 15, printed pp. 40-48, 217-235.

Daily commands, with examples

Use the long object-oriented spelling when it helps you remember what is being managed.

PURPOSE	EXAMPLE	RESULT / NOTE
Check connection	docker version	Shows client and server information.
Inspect environment	docker info	Engine configuration and runtime details.
Pull image	docker image pull alpine	Downloads the referenced image.
List images	docker image ls	Lists local image references.
Inspect image	docker image inspect alpine	Shows metadata, including configuration.
Build	docker build -t web-guide:1.0 .	Builds using the current directory context.
Tag	docker image tag web-guide:1.0 web-guide:demo	Adds another local reference.
Remove image	docker image rm web-guide:demo	Removes a reference; content may remain in use.
Interactive task	docker run --rm -it alpine sh	Starts a shell; removes container on exit.
Background task	docker run -d --name web nginx:alpine	Creates a detached web server.
List containers	docker container ls -a	Includes running and stopped containers.
Read logs	docker logs --tail 50 web	Reads recent captured stdout/stderr.
Inspect runtime	docker inspect web	Shows mounts, networks, state and config.
Resource usage	docker stats --no-stream web	One resource-usage snapshot.
New shell	docker exec -it web sh	Creates a process in a running container.
Stop / start	docker stop web docker start web	Stops gracefully / restarts the same container.
Remove	docker stop web docker rm web	Deletes the container after it stops.

Examples are independent snippets unless a lab states an execution order. Replace names to match your own environment. Image tags in exercises are convenient mutable references, not exact reproducibility pins.

Apps, storage and clusters

These examples extend the concepts and names used in this guide.

PURPOSE	EXAMPLE	RESULT / NOTE
Create bridge	docker network create app-net	Creates a user-defined local bridge.
Inspect network	docker network inspect app-net	Displays network endpoints and settings.
Connect container	docker network connect app-net web	Attaches an existing container.
Create volume	docker volume create notes	Allocates named persistent storage.
List volumes	docker volume ls	Lists Docker-managed volumes.
Inspect volume	docker volume inspect notes	Shows driver and storage metadata.
Validate Compose	docker compose config	Resolves and checks the application model.
Run project	docker compose up -d	Creates/updates and starts the project.
Observe project	docker compose ps docker compose logs --tail 30	Status and recent output.
Stop project	docker compose down	Removes project containers and networks.
Cluster status	docker node ls	Manager command; requires Swarm.
Inspect service	docker service ps class-site	Shows tasks and scheduling state.
Scale service	docker service scale class-site=3	Changes the desired replica count.
Revert service	docker service rollback class-site	Returns to the prior service configuration.
Deploy stack	docker stack deploy -c stack.yaml class-app	Deploys the saved stack definition.
Inspect stack	docker stack services class-app	Shows the services belonging to the stack.

Cleanup with intent

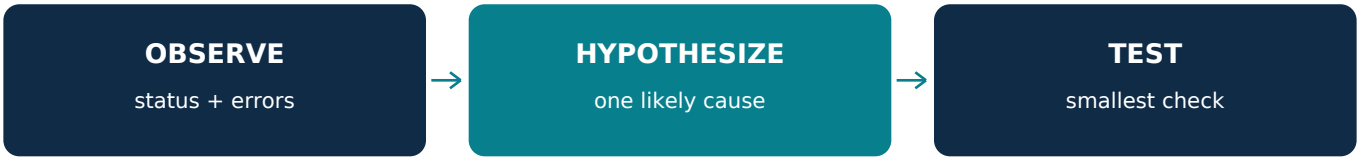
Removing containers, volumes, services or stacks changes real resources. Check names first. Volume deletion and compose down -v can destroy data; broad prune commands are deliberately unnecessary for these labs.

Diagnose from evidence

Check state first, then application output, networking and storage.

SYMPTOM	FIRST CHECK	LIKELY EXPLANATION / NEXT STEP
Cannot reach daemon	docker version	Check Engine status, socket access and the selected context.
Container exits immediately	docker container ls -a docker logs NAME	The main process ended or failed; inspect exit state and command.
Browser cannot connect	docker port NAME docker logs NAME	Confirm the host mapping and that the app listens on the correct container interface/port.
Port already allocated	docker container ls	Another process owns the host port. Choose another host port or stop its owner.
Name already in use	docker container ls -a	Stopped containers still reserve their names. Reuse, rename or remove the old container.
Cannot reach another container	docker network inspect NET	Check shared network, DNS name and service listening address. localhost points to self.
File disappeared	docker inspect NAME	Data may have been in the removed writable layer; check mounts and paths.
Swarm task is pending	docker service ps SERVICE	Check task errors, available resources, placement rules and image access.
Rescheduled task lacks data	docker service ps SERVICE	A node-local volume did not move with the task. Review storage architecture.
Swarm cannot change state	docker node ls	Check manager connectivity and quorum.

A repeatable investigation loop



Change one thing, rerun the check, and explain why the evidence supports the conclusion.

An unhealthy status and an exited process are different signals. Read the health-check result, process state and service controller behavior before assuming a restart should occur.

Source synthesis: book chapters 7, 9-15.

A 35-minute practical lesson

Four small experiments build confidence without requiring application source code.

TIME	ACTIVITY	SUCCESS CHECK
0-5 min	Check docker version. Pull nginx:alpine and inspect it.	Identify the image and its default startup configuration.
5-15 min	Build the two-file website on page 7 and run it.	curl returns your HTML from localhost:8080.
15-25 min	Complete the volume lab on page 11.	A new container reads a file written by the first.
25-35 min	Run the Compose example on page 9.	redis-cli ping returns PONG; down preserves the volume.

Make persistence visible in Compose

RUN IN THE PAGE 9 PROJECT DIRECTORY

```
docker compose exec redis redis-cli SET lesson docker
docker compose down
docker compose up -d
docker compose exec redis redis-cli GET lesson
```

Expected result: docker. Redis append-only persistence and the named volume preserve the value across container recreation. This proves the configured storage path; it does not test disaster recovery.

Three explain-back prompts

Before running: Predict which ports exist inside the containers and which are published on the host.

After stopping: Explain why an image is still present after its container is removed.

After recreation: Explain why the Redis value survived and name the command that would delete its volume.

> **Finish the lesson**
> Run docker compose down to stop the project. Keep the named volume if you want to revisit the saved value. Delete it only when the practice data is no longer useful.

Test your mental model

Answer without looking, then use the compact answer key.

QUESTION	YOUR TARGET IDEA
1. Does stopping a container delete its writable layer?	Distinguish stopped from removed.
2. Does EXPOSE 80 make a web app reachable from the host?	Distinguish metadata from publication.
3. Does latest mean the newest or safest version?	Distinguish labels from guarantees.
4. What does localhost mean inside a container?	Identify the network namespace.
5. Why use a volume for application data?	Separate storage and container lifecycles.
6. Can five workers compensate for two lost managers in a three-manager Swarm?	Separate worker capacity from manager quorum.
7. Does Compose startup order guarantee readiness?	Separate process start from application health.
8. Does a node-local volume follow a rescheduled Swarm task?	Separate local and shared storage.
9. Why use a multi-stage Dockerfile?	Separate build requirements and runtime requirements.
10. Does an unhealthy standalone container automatically restart?	Separate health status and exit behavior.

Answer key

1. No; removal discards it. **2.** No; use a published port such as -p. **3.** No; latest is a mutable tag. **4.** The container itself. **5.** Data can survive container removal. **6.** No; workers do not vote, and one of three managers cannot form a majority. **7.** No; readiness needs additional checks and retries. **8.** No; use an appropriate storage design. **9.** To copy only required artifacts into the final image. **10.** No; restart policies normally respond to process exit, while health status is separate.

> **Mastery task**
> Draw source -> image -> container. Add one network and one volume. Explain which objects remain after the container is removed, and which component would recreate a failed task in Swarm.

Teach it in 90 minutes

A classroom outline built around prediction, demonstration and explanation.

TIME	TEACH / DEMONSTRATE	ASK LEARNERS TO...
0-10	Image, container and VM comparison; pages 3-4.	Explain the shared-kernel model.
10-25	Run, observe, stop, start, remove; page 6.	Predict state after each command.
25-40	Build the static website; pages 7-8.	Explain build time versus runtime.
40-55	Network names and port mapping; page 10.	Trace a browser request to the container.
55-65	Volume persistence experiment; page 11.	Recreate a container and prove data survival.
65-75	Compose project; page 9.	Identify services, network and named volume.
75-85	Swarm and quorum chart; pages 12-13.	Explain desired state and manager majority.
85-90	Exit quiz; page 21.	Explain one misconception in their own words.

Three drawings worth repeating

Artifact flow: code + Dockerfile -> image -> containers. Point out that many containers can come from one image.

Network path: browser -> host port -> container port. Then draw a separate path between containers using a shared network and a DNS name.

Data lifecycle: two successive containers -> one named volume. Erase the first container while leaving the volume on the board.

> **Assess explanation, not typing speed**
> A learner is ready to move on when they can predict the effect of a command, select evidence to verify it, and explain the result. For advanced classes, add the overlay and stack exercises on pages 14-15.

Follow-up: repeat the labs from memory and explain one failure using logs and metadata.

Read the edition in context

The foundation is the supplied book; selected updates prevent historical details from confusing practice.

Primary source: Nigel Poulton, Docker Deep Dive: Zero to Docker in a single book, May 2020 edition. Topic references throughout this guide point to the printed page labels in the supplied PDF.

BOOK COVERAGE	WHERE THIS GUIDE CONDENSES IT
Ch. 1-4: big picture and installation	Guide 2-4. Platform-specific 2020 installation steps are omitted.
Ch. 5-8: Engine, images, containers, builds	Guide 4-8 and daily command reference.
Ch. 9-14: Compose, Swarm, networks, storage, stacks	Guide 9-15 and infrastructure command reference.
Ch. 15-16: security and further learning	Guide 16 and practice / teaching pages 19-22.

Modernization notes - checked 16 October 2026

[1] Compose CLI: the book uses the Python-era docker-compose command. This guide uses docker compose. See Docker: Compose history.

[2] Compose file version: modern Compose treats the top-level version field as obsolete, so page 9 omits it. See Docker: version and name.

[3] Stack compatibility: docker stack deploy still uses the legacy Compose v3 format; page 15 intentionally uses version 3.8. See Docker: deploy a stack to a Swarm.

[4] Image trust: the book's Docker Content Trust / Notary v1 workflow is historical. Docker documents retirement and alternatives; no old signing lab is included. See Docker: content trust.

[5] Kubernetes: the book's direct Docker integration reflects its era. Kubernetes removed its built-in dockershim in v1.24; images built using Docker remain usable with compatible runtimes. See Kubernetes: dockershim removal FAQ.

> **Scope of this companion**
> Diagrams are conceptual; the quorum chart uses the stated majority formula rather than benchmark data. Labs are newly written examples. This condensed guide supports revision and instruction; use the book for full explanations and official docs for environment-specific installation and deployment details.